```
"""
residual_comparator.py — GRUS Audit Pipeline Stage 6
Specification: GRUS-STD-001-v1.0 Section 3 Stage 6; Clause 3
Parses verify.py stdout for residual reports and compares each against
the manifest's declared values within tolerance.
Published by: Green Recursive Utility Service LLC
"""
import re
from typing import Dict, List
from dataclasses import dataclass, field

TOLERANCE_PCT = 1.0
TOLERANCE_SIGMA = 0.05

@dataclass
class ResidualComparisonResult:
    passed: bool
    matched: List[Dict] = field(default_factory=list)
    mismatched: List[Dict] = field(default_factory=list)
    failure_reason: str = ""

PCT_PATTERN = re.compile(r"residual\s*[:=]?\s*([\d.eE+-]+)\s*%", re.IGNORECASE)
SIGMA_PATTERN = re.compile(r"residual\s*[:=]?\s*([\d.eE+-]+)\s*sigma", re.IGNORECASE)
DOMAIN_PATTERN = re.compile(r"^\s*(D\d+|C\d+|P\d+)[\s/].*", re.IGNORECASE)

def parse_residuals_from_stdout(stdout: str) -> List[Dict]:
    residuals = []
    current_domain = None
    for line in stdout.splitlines():
        m_dom = DOMAIN_PATTERN.match(line)
        if m_dom:
            current_domain = m_dom.group(1).upper()
        m_pct = PCT_PATTERN.search(line)
        m_sigma = SIGMA_PATTERN.search(line)
        if m_pct:
            try:
                residuals.append({"domain": current_domain or "unknown",
                                  "value": float(m_pct.group(1)), "unit": "%"})
            except ValueError: pass
        elif m_sigma:
            try:
                residuals.append({"domain": current_domain or "unknown",
                                  "value": float(m_sigma.group(1)), "unit": "sigma"})
            except ValueError: pass
    return residuals

def compare_residuals(manifest: Dict, stdout: str) -> ResidualComparisonResult:
    parsed = parse_residuals_from_stdout(stdout)
```

```
    declared = manifest.get("predictions", [])
    matched, mismatched = [], []
    for d in declared:
        if d.get("status") != "confirmed": continue
        d_id = d.get("id", "")
        d_residual_str = str(d.get("residual", "")).strip()
        m_pct = re.search(r"([\d.eE+-]+)\s*%", d_residual_str)
        m_sigma = re.search(r"([\d.eE+-]+)\s*sigma", d_residual_str, re.IGNORECASE)
        match_found = False
        for parsed_r in parsed:
            if parsed_r["domain"].upper() in d_id.upper() or d_id.upper() in
parsed_r["domain"].upper():
                if m_pct and parsed_r["unit"] == "%":
                    if abs(parsed_r["value"] - float(m_pct.group(1))) <= TOLERANCE_PCT:
                        matched.append({"id": d_id, "declared": d_residual_str,
                                        "observed": f"{parsed_r['value']:.3f}%"})
                        match_found = True; break
                elif m_sigma and parsed_r["unit"] == "sigma":
                    if abs(parsed_r["value"] - float(m_sigma.group(1))) <= TOLERANCE_SIGMA:
                        matched.append({"id": d_id, "declared": d_residual_str,
                                        "observed": f"{parsed_r['value']:.3f}σ"})
                        match_found = True; break
        if not match_found:
            mismatched.append({"id": d_id, "declared": d_residual_str,
                               "reason": "no matching residual found in verify.py output"})
    if mismatched:
        return ResidualComparisonResult(passed=False, matched=matched, mismatched=mismatched,
            failure_reason=f"{len(mismatched)} declared residuals do not match verify.py
output")
    return ResidualComparisonResult(passed=True, matched=matched, mismatched=[])

if __name__ == "__main__":
    import sys, yaml, os
    if len(sys.argv) < 3:
        print("Usage: python residual_comparator.py <repo_path> <verify_stdout_file>");
sys.exit(0)
    with open(os.path.join(sys.argv[1], "GRUS_MANIFEST.yaml")) as f:
        m = yaml.safe_load(f)
    with open(sys.argv[2]) as f:
        stdout = f.read()
    r = compare_residuals(m, stdout)
    if r.passed:
        print(f"[PASS] {len(r.matched)} residuals matched")
    else:
        print(f"[FAIL] {r.failure_reason}")
        for mm in r.mismatched: print(f"  {mm['id']}: {mm['reason']}")
        sys.exit(1)
```